

Comunicação entre Pods e Services

O que você aprenderá nesta página

- Explicar como os Pods se comunicam dentro do cluster
- Criar Services para estabilizar acesso a aplicações
- Diferenciar ClusterIP, NodePort, LoadBalancer e ExternalName

Quando trabalhamos apenas com um Deployment, cada Pod criado recebe seu próprio endereço IP. Esse IP, porém, não deve ser usado como contrato de comunicação: Pods são substituídos, recriados e movidos entre nós. A solução do Kubernetes para esse problema é o Service, um recurso que oferece um ponto de acesso estável para um conjunto de Pods selecionados por labels.

O Service separa quem consome a aplicação de onde os Pods estão rodando naquele momento. Em vez de chamar diretamente o IP de um Pod, outro componente chama o nome DNS do Service, como `backend-svc.default.svc.cluster.local`. O Kubernetes mantém os Endpoints atualizados conforme os Pods entram ou saem do conjunto selecionado.

Labels, selectors e Endpoints

Um Service encontra seus Pods por meio de seletores. Por isso, labels bem definidas deixam de ser detalhe estético e viram parte da arquitetura da aplicação. Se o seletor do Service não combinar com os labels do Pod, o Service existirá, mas não encaminhará tráfego para lugar nenhum.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-dep
spec:
  replicas: 3
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
        - name: app
          image: nginx:1.27
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: app-svc
spec:
  selector:
    app: app
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Depois de aplicar o manifesto, os comandos `kubectl get svc`, `kubectl get endpoints app-svc` e `kubectl describe svc app-svc` ajudam a verificar se o Service realmente

encontrou os Pods.

Tipos de Service

O tipo `ClusterIP` é o padrão e expõe a aplicação apenas dentro do cluster. Ele é ideal para comunicação interna entre microsserviços. O `NodePort` abre uma porta em todos os nós do cluster e encaminha o tráfego para o Service. O `LoadBalancer` solicita ao provedor de nuvem um balanceador externo. O `ExternalName` cria um alias DNS para um serviço externo.

Headless Service

Um `Headless Service` é criado com `clusterIP: None`. Nesse caso, o Kubernetes não fornece um IP virtual único para balanceamento. Em vez disso, o DNS retorna os endereços dos Pods diretamente. Esse padrão é importante para aplicações que precisam conhecer a identidade individual de cada réplica, como bancos de dados e sistemas distribuídos.

```
apiVersion: v1
kind: Service
metadata:
  name: postgres-headless
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
```

Boas práticas

Use nomes claros, mantenha labels consistentes e evite depender de IPs de Pods. Para comunicação interna, comece com `ClusterIP`. Para publicação externa, prefira `Ingress`, `Gateway API` ou um `LoadBalancer` controlado. Sempre valide se os `Endpoints` aparecem.